Davante

16

BASE DE DATOS

Tipos de datos compuestos y cursores

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Registros 5

/ 3. Ejemplos de uso y asignación de registros 5

/ 4. Arrays: Definición y sintaxis 6

/ 5. Asignación de arrays y métodos 7

/ 6. Arrays asociativos 8

/ 7. Tablas anidadas 9

/ 8. Cursores 10

8.1. Concepto 10

8.2. Propiedades de los cursores 10

/ 9. Tipos de cursores 11

9.1. Cursores implícitos 11

9.2. Cursores explícitos 12

9.3. Cursores explícitos con parámetros 12

/ 10. Ciclo de vida de los cursores explícitos 12

10.1. Apertura de cursores 12

10.2. Extracción de datos 13

10.3. Cierre de cursores 13

ÍNDICE

/ 11. Actualización de registros con cursores 13

/ 12. Caso práctico 1: “Colección de nombres” 14

/ 13. Caso práctico 2: “Cursor sobre las noticias” 15

/ 14. Resumen y resolución del caso práctico de la unidad 16

/ 15. Bibliografía 17

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Entender el concepto de registro y cómo utilizarlo.

Conocer el concepto de array, sus posibles aplicaciones y cómo implementarlos en PL/SQL.

Explorar el concepto de tabla anidada en PL/SQL, sus usos y cómo se implementan.

Comprender los cursores y estudiar su uso e implementación en PL/SQL.

/ 1. Introducción y contextualización práctica

En PL/SQL, así como en otros lenguajes de programación complejos, existen varios elementos útiles para llevar a cabo acciones específicas. Entre ellos, se encuentran los registros, *arrays* y tablas anidadas, que se conocen como tipos de datos compuestos.

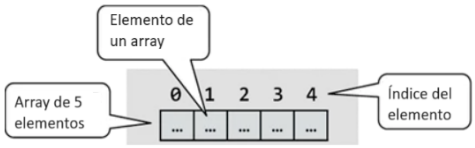


Fig.1. Ejemplo de array de cinco elementos.

Estas estructuras de datos compuestos son especialmente valiosas para resolver problemas más complejos, donde los tipos de datos simples no son suficientes.

Por otro lado, los cursores son herramientas importantes para manejar conjuntos de resultados de consultas SQL, permitiendo trabajar con los datos de manera ordenada y realizar operaciones específicas sobre ellos.

En este tema, abordaremos en detalle en qué consisten todos estos conceptos y cómo se implementan en PL/SQL. Asimismo, proporcionaremos ejemplos prácticos para ilustrar su uso y funcionamiento.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Explorando las estructuras fundamentales en programación y bases de datos"

https://on.soundcloud.com/T5EXbvW5RzacgyPv8

/ 2. Registros

Los registros se pueden describir como un **conjunto de datos relacionados, organizados en campos**. Un ejemplo sería el registro HABITANTE, con los campos ID, Nombre y Domicilio.

Cada campo de un registro puede contener diferentes tipos de datos.

Un ejemplo de registro podría ser también el de la figura 2.

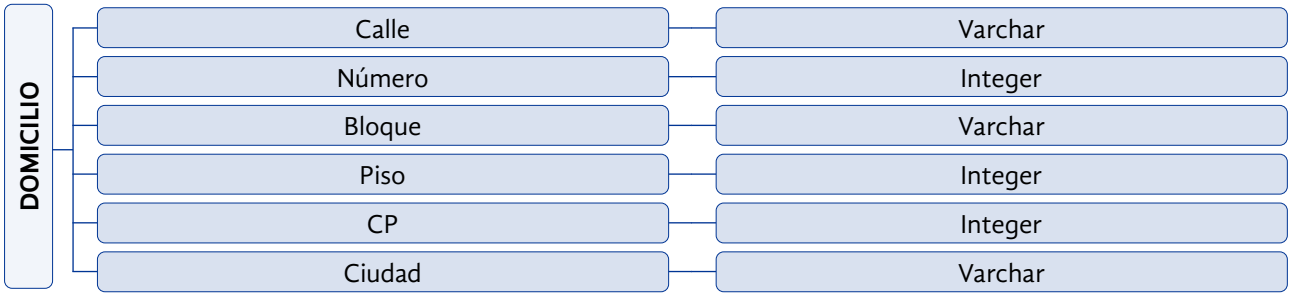


Fig.2. Ejemplo de registro.

Estas estructuras son comunes en los lenguajes de programación. Según la definición mencionada, los registros son **similares a las filas o registros de una tabla**, lo cual explica su amplia utilización en PL/SQL.

La **sintaxis** para declarar un registro en PL/SQL se muestra en la figura 3.

```
TYPE registro IS RECORD (
    Campo1 TipoDatos1 [[NOT NULL] := Expresion1],
    Campo2 TipoDatos2 [[NOT NULL] := Expresion2],
    ...
    CampoN TipoDatosN [[NOT NULL] := ExpresionN]
);
```

Fig.3. Sintaxis de un registro.

Para **acceder a los datos de un campo específico** dentro de un registro, se utiliza la sintaxis siguiente: **NombreRegistro.Campo**.

/ 3. Ejemplos de uso y asignación de registros

En este apartado veremos algunos ejemplos de cómo crear, insertar y asignar registros.

En el ejemplo de la figura 4, implementado en Oracle SQL Developer, se **crea un registro** y se implementa un fragmento de código en PL/SQL para **insertar el registro** en la tabla PRUEBA de la base de datos.

```
DECLARE
    TYPE Persona IS RECORD(
        CODIGO INTEGER(2),
        NOMBRE VARCHAR(20),
        EDAD NUMBER(3)
    );
    Juan Persona;
BEGIN
    Juan.CODIGO := 1;
    Juan.NOMBRE := 'Juan';
    Juan.EDAD := 33;
    INSERT INTO PRUEBA VALUES Juan;
END;
```

Fig.4. Ejemplo de creación de un registro.

Se pueden utilizar los siguientes **atributos**:

- **%TYPE**: Especifica que el tipo de dato de un campo en un nuevo registro será el mismo que el de un campo de una tabla existente.
- **%ROWTYPE**: Indica que un registro tiene la misma estructura que una fila completa de una tabla.

En PL/SQL, **se pueden asignar registros si son del mismo tipo, pero no si son de tipos diferentes**; en ese caso, es **necesario asignar los valores campo por campo**. Además, **no es posible realizar comparaciones directas entre registros**, incluso si son del mismo tipo.

```
DECLARE
TYPE Persona IS RECORD(
    CODIGO INTEGER(2),NOMBRE VARCHAR(20),EDAD NUMBER(3)
    );

TYPE Persona_aux IS RECORD(
    CODIGO INTEGER(2),NOMBRE VARCHAR(20),EDAD NUMBER(3)
    );
Juan Persona;
Pepe Persona_aux;
SUBTYPE Persona_prueba IS PRUEBA%ROWTYPE;
Ana Persona_prueba;

BEGIN
Juan := Ana; --Sería permitido
Juan := Pepe; -- No estaría permitido
SELECT * INTO Juan From PRUEBA; -- Se podría realizar
END;
```

Fig.5. Ejemplos de asignación de registros.

Como se puede observar en el ejemplo anterior, el de la figura 5, **es posible ejecutar consultas SELECT sobre los registros, pero no lo es realizar operaciones INSERT directamente en ellos**.

Vídeo 1. "Uso y asignación de registros en Oracle Developer"

https://bit.ly/3oauO0k

/ 4. Arrays: Definición y sintaxis

Los arrays son estructuras comunes en los lenguajes de programación. Consisten en **una colección de elementos del mismo tipo organizados en un orden específico**, lo que **permite acceder a cada uno de ellos mediante un índice**.

En PL/SQL, se utiliza la expresión **VARRAY** para **crear y referenciar arrays**. Al crear un array, es **necesario definir su capacidad máxima**, que establece el **número máximo de elementos que puede contener**. El array debe tener al menos un elemento, aunque no hay un límite superior específico para su tamaño.

La sintaxis para **declarar un array** se muestra en la figura 6.

```
TYPE nombreArray
IS VARRAY (capacidad_máxima)
OF tipo_de_elementos [NOT NULL];
```

Fig.6. Sintaxis para declarar un array.

Los elementos del array pueden ser de **cualquier tipo de datos** permitido en PL/SQL, **a excepción de** tipos como **BOOLEAN**, **LONG**, **NATURAL**, **BINARY INTEGER**, entre otros.

Es importante recordar que, **al definir un array, este se crea vacío**, por lo que es necesario agregar los elementos deseados después de su creación.

En la figura 7, se muestra un ejemplo de cómo **crear e inicializar un array**:

```
TYPE persona_prueba IS VARRAY(100) OF Persona;
Ana persona_prueba := persona_prueba(Persona(4,'Ana',45));
Prueba persona_prueba :=persona_prueba();
```

Fig.7. Ejemplo de creación e inicialización de un array.

En el ejemplo anterior, se ha creado un array con una capacidad de hasta cien elementos de tipo PERSONA, y se le han añadido varios elementos.

Se puede **acceder a cualquier elemento de un array** utilizando la sintaxis: **NombreArray (ÍndiceElemento)**.

/ 5. Asignación de arrays y métodos

Al igual que con los registros, **es posible asignar un VARRAY a otro siempre que ambos sean del mismo tipo**.

En la figura 8, se muestra un ejemplo de ello:

```
DECLARE
    TYPE tipo1 IS VARRAY(20) OF NUMBER;
    TYPE tipo2 IS VARRAY(20) OF NUMBER;
    una_prueba1 tipo1 := tipo1(); --Elemento array tipo1
    una_prueba2 tipo2 := tipo2(); --Elemento array tipo2
    otra_prueba1 tipo1 := tipo1(); --Elemento array tipo1
BEGIN
    otra_prueba1 := una_prueba1; --Se puede realizar
    una_prueba1 := una_prueba2; --No se puede realizar
END;
```

Fig.8. Ejemplo de asignación de un VARRAY a otro.

Además, existen ciertos **métodos** (también llamados **atributos**) **que son muy útiles para manipular arrays** en PL/SQL y otros lenguajes de programación:

- **EXTEND(a, b)**: Se usa para extender un array, agregando copias de un elemento específico del mismo. Al emplear esta función, se añaden 'a' copias del elemento ubicado en la posición 'b' del array (el índice 'b').

  En la figura 9, se muestra un ejemplo de uso de EXTEND.

```
una_prueba1.EXTEND(); -- Se añade un elemento nulo
una_prueba1.EXTEND(5); -- Se añaden 5 elementos nulos
una_prueba1.EXTEND(5,2); -- Se añaden 5 copias del elemento 2
```

Fig.9. Ejemplo de uso de EXTEND en el array una_prueba1.

- **COUNT**: Permite conocer el número de elementos presentes en un array en un momento dado. En la figura 10, se muestra un ejemplo de uso de COUNT.

```
una_prueba1.COUNT;
```

Fig.10. Ejemplo de uso de COUNT en el array una_prueba1.

- **LIMIT**: Muestra la capacidad o tamaño máximo que puede alcanzar un array. En la figura 11, se muestra un ejemplo de ello:

```
una_prueba1.LIMIT; --Indica la capacidad (20)
```

Fig.11. Ejemplo de uso de LIMIT en el array una_prueba1.

- **FIRST**: Indica el primer elemento del array.
- **LAST**: Señala el último elemento del array.
- **PRIOR**: Devuelve el elemento anterior al referenciado.
- **NEXT**: Devuelve el elemento siguiente al referenciado.

Enlaces de interés...

En este enlace, encontrarás la documentación oficial de Oracle en la que podrás ampliar tus conocimientos sobre los tipos de datos compuestos y colecciones disponibles en este potente SGBD.

/ 6. Arrays asociativos

También conocidos como '**index-by tables**', son una **colección de pares clave-valor donde las claves pueden ser tanto numéricas como cadenas de caracteres**. En la tabla 1 se muestran las principales **características** de los arrays asociativos.

Característica	Descripción
Tamaño	No tienen un límite en el número de elementos que pueden contener y pueden cambiar de tamaño dinámicamente.
Índices	Las claves pueden ser de tipo BINARY_INTEGER o VARCHAR2.
Uso	Ideales para búsquedas rápidas y dinámicas de datos usando claves personalizadas.
Persistencia	Solo existen en memoria y no pueden ser almacenados directamente en la base de datos.

Tabla 1. Características de los arrays asociativos.

Los arrays asociativos se **definen** a través de la sentencia que se muestra en la figura 12.

```
DECLARE
  TYPE t_poblacion_ciudad
    IS TABLE OF NUMBER INDEX BY VARCHAR2(100);
  v_poblacion_ciudad t_poblacion_ciudad;
```

Fig.12. Definición y declaración de array asociativo.

Para **inicializar sus valores, se utiliza el nombre de la variable seguido de la clave entre paréntesis y luego se le asigna el valor**. En la figura 13 se muestra un ejemplo.

```
v_poblacion_ciudad('Madrid') := 3223000;
v_poblacion_ciudad('Barcelona') := 1620000;
v_poblacion_ciudad('Valencia') := 800000;
v_poblacion_ciudad('Sevilla') := 700000;
```

Fig.13. Inicialización de arrays asociativos.

Para **acceder a sus valores**, la sintaxis consistiría en **utilizar el nombre de la tabla seguido del valor de la clave entre paréntesis**. En la figura 14 se muestra un ejemplo.

```
DBMS_OUTPUT.PUT_LINE('Población de Madrid: ' || v_poblacion_ciudad('Madrid'));
DBMS_OUTPUT.PUT_LINE('Población de Barcelona: ' || v_poblacion_ciudad('Barcelona'));
DBMS_OUTPUT.PUT_LINE('Población de Valencia: ' || v_poblacion_ciudad('Valencia'));
DBMS_OUTPUT.PUT_LINE('Población de Sevilla: ' || v_poblacion_ciudad('Sevilla'));
```

Fig.14. Consulta de valores en un array asociativo.

/ 7. Tablas anidadas

Al igual que los arrays, son **colecciones de elementos del mismo tipo**. Sin embargo, a diferencia de ellos, **no tienen un límite fijo de elementos**, lo que significa que **su tamaño es dinámico**. La **sintaxis** para declarar una tabla anidada se muestra en la figura 15.

```
TYPE nombreTabla IS TABLE
OF  tipo_de_elementos [NOT NULL];
```

Fig.15. Sintaxis para la declaración de una tabla anidada.

Al igual que en los arrays, una tabla anidada **se inicializa vacía**, por lo que es necesario agregar elementos para su utilización.

Además, hay algunos **métodos** que pueden emplearse con tablas anidadas:

- **EXTEND(a, b)**: Añade 'a' elementos copiados desde el índice 'b'.
- **COUNT**: Indica el número de elementos presentes.
- **FIRST** y **LAST**: Devuelven el primer y último elemento, respectivamente.
- **PRIOR** y **NEXT**: Devuelven el elemento anterior y siguiente al actual, respectivamente.
- **TRIM(a)**: Elimina los últimos 'a' elementos de la tabla.
- **DELETE(a, b)**: Borra los elementos desde el índice 'a' hasta el 'b'. Si se proporciona solo un argumento, elimina el elemento en ese índice.

En tablas anidadas, el método **LIMIT no se aplica**, ya que no hay un tamaño máximo predefinido.

En la figura 16, se presenta un ejemplo que muestra diferentes operaciones sobre una tabla anidada y los resultados obtenidos en cada caso:

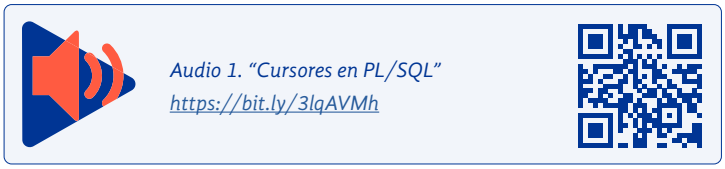
```
DECLARE
  TYPE numbers IS TABLE OF NUMBER;
  table_numbers numbers := numbers();
  num NUMBER;
BEGIN
  num := table_numbers.COUNT; -- Debería devolver cero
  FOR a IN 1..100 LOOP
    table_numbers.EXTEND;
    table_numbers(a) := a;
  END LOOP;

  num := table_numbers.COUNT; -- Ahora valdría 100
  table_numbers.DELETE(100);
  num := table_numbers.LAST; -- Su valor sería 99
  num := table_numbers.FIRST; -- Su valor sería 1
  table_numbers.DELETE(1);
  num := table_numbers.FIRST; -- Su valor sería 2
END;
```

Fig.16. Ejemplo de uso de tabla anidada.

/ 8. Cursores

Son herramientas esenciales en los sistemas de bases de datos. En este apartado, exploraremos su naturaleza y cómo se aplican en distintos contextos



Audio 1. "Cursores en PL/SQL"
https://bit.ly/3lqAVMh

8.1. Concepto

Los **cursores** son estructuras que se asemejan a las variables, ya que **pueden contener datos e información generados a partir de una consulta**. Sin embargo, a diferencia de las variables, los datos en los cursores **pueden ser recorridos** de manera secuencial, similar a un array.

Así, el uso de cursores es especialmente útil cuando el resultado de una consulta consiste en múltiples filas de una tabla.

8.2. Propiedades de los cursores

Los cursores cuentan con cuatro propiedades que permiten controlar su estado y su información:

- **%FOUND**: Devuelve un **TRUE**, si el último acceso al cursor mediante la sentencia **FETCH** (que explicaremos a lo largo de este tema) ha devuelto una fila, o **FALSE**, en caso de no haber devuelto resultados.

- **%NOTFOUND**: Es justamente el caso contrario a %FOUND. Devuelve un **TRUE** si la última consulta al cursor con **FETCH** no ha devuelto resultados. Devuelve **FALSE** en caso de que sí se hayan encontrado resultados. En la figura 17 se muestra un ejemplo del uso de esta propiedad.

```
EXIT WHEN cursor_emp%NOTFOUND;
```

Fig.17. Ejemplo de NOTFOUND.

- **%ISOPEN**: Permite verificar el estado del cursor. Devolverá un valor **TRUE** en caso de que el cursor esté abierto y listo para leer resultados; en cambio, devolverá **FALSE** en caso de que el cursor se encuentre cerrado. Gracias a esta propiedad se pueden evitar errores al acceder a un cursor cerrado. En la figura 18 podemos ver un ejemplo de su uso.

```
IF NOT cursor_emp%ISOPEN THEN
    OPEN cursor_emp;
END IF;
```

Fig.18. Ejemplo de ISOPEN.

- **%ROWCOUNT**: Permite comprobar el número de filas que el cursor ha procesado mediante **FETCH** hasta el momento. En la figura 19 se muestra un ejemplo.

```
WHILE cursor_emp%FOUND LOOP
  DBMS_OUTPUT.PUT_LINE('Procesadas: ' || cursor_emp%ROWCOUNT);
```

Fig.19. Ejemplo de ROWCOUNT.

9. Tipos de cursores

Existen dos tipos diferenciados de cursores, los **explícitos**, que son definidos por el usuario en un momento dado, y los **implícitos**, que los define el SGBD.

9.1. Cursores implícitos

Se caracterizan por ser creados automáticamente por el SGBD a la hora de ejecutar ciertas **consultas, como SELECT INTO, INSERT, UPDATE y DELETE, bajo el nombre 'SQL'**. En la figura 20 se muestra el uso de las propiedades FOUND y ROWCOUNT del cursor implícito creado al actualizar datos.

```
BEGIN
    UPDATE emp SET sal = sal * 1.1 WHERE deptno = 10;
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' filas actualizadas.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No se han actualizado filas.');
    END IF;
END;
```

Fig.20. Ejemplo de cursor implícito.

9.2. Cursores explícitos

Estos deben ser **definidos y gestionados por el usuario**, y se utilizan para **recorrer los resultados de consultas SELECT que devuelven varias filas**. Han de ser definidos en la sección DECLARE del *script*. En la figura 21 se muestra un ejemplo de la sintaxis necesaria para su definición:

```
DECLARE
    CURSOR emp_cursor IS
        SELECT ename, job FROM emp;
```

Fig.21. Ejemplo de definición de un cursor explícito.

9.3. Cursores explícitos con parámetros

Los cursores explícitos con parámetros son aquellos que, en su definición, **incluyen parámetros que pueden ser utilizados en la consulta SQL para filtrar o modificar los resultados basados en valores específicos**.

En la figura 22 se muestra un ejemplo de definición de un cursor con parámetros para acotar los resultados de la consulta en base al número de departamento:

```
DECLARE
    CURSOR emp_cursor (p_deptno NUMBER) IS
        SELECT ename, job FROM emp
        WHERE deptno = p_deptno;
```

Fig.22. Ejemplo de definición de cursor explícito con parámetros.



Vídeo 2. "Ejemplo de uso de cursores implícitos y explícitos"

https://bit.ly/37qCvsP



/ 10. Ciclo de vida de los cursores explícitos

Para asegurar el correcto uso de los cursores, es necesario manejar correctamente su ciclo de vida. Ahora que ya hemos visto cómo definirlos, veremos cómo realizar su **apertura**, su **extracción de datos** y el **cierre**.

10.1. Apertura de cursores

Esta es la fase en la que **se inicializa con los datos obtenidos al ejecutar la consulta que lo define**. La apertura se realiza a través de la instrucción '**OPEN nombreCursor**', como se muestra en la figura 23.

```
BEGIN
    OPEN emp_cursor;
```

Fig.23. Apertura de cursor sin parámetros.

En caso de que el cursor utilice **parámetros**, debemos indicarlo como se muestra en la figura 24.

```
BEGIN
    OPEN emp_cursor(10);
```

Fig.24. Apertura de cursor con parámetros.

10.2. Extracción de datos

Una vez que el cursor está abierto con los datos obtenidos de ejecutar la consulta que lo define, estos deben ser **recorridos**. Para ello utilizaremos la instrucción '**FETCHnombreCursorINTOnombreVariable**'. En la figura 25 se muestra un ejemplo de su sintaxis.

```
FETCH emp_cursor INTO v_ename, v_job;
```

Fig.25. Ejemplo FETCH de datos del cursor sobre dos variables.

Como se puede ver, se asignan dos variables para almacenar la información devuelta por FETCH. Esto se debe a que la consulta del cursor selecciona dos columnas de la tabla. También podría utilizarse un registro formado por los dos campos.

10.3. Cierre de cursores

La última etapa en el ciclo de vida de los cursores es el **cierre** de estos y la **liberación de los recursos** que tenían reservados para almacenar los datos obtenidos en la ejecución de la consulta.

El cierre se realiza a través de la instrucción **CLOSE**. En la figura 26 podemos ver un ejemplo de su uso.

```
CLOSE emp_cursor;
```

Fig.26. Ejemplo del cierre de un cursor.

Enlaces de interés...

En el siguiente enlace, podrás encontrar un vídeo con algunos ejemplos sobre el uso de cursores en PLSQL combinándolos con estructuras de control, que te permitirá tener una perspectiva más amplia sobre los cursores: https://www.youtube.com/watch?v=VXmPNwz8YSA

/ 11. Actualización de registros con cursores

En PLSQL es posible llevar a cabo una **actualización de los registros que se han obtenido como resultado de ejecutar la consulta que define un cursor**. Sin embargo, la gran cantidad de datos que se pueden obtener conlleva que el tiempo empleado para recorrerlos y actualizarlos pueda extenderse y, por tanto, darse la situación de que esos registros sean modificados por otro usuario.

Para poder realizar las actualizaciones de registros mediante cursores, es **conveniente** llevar a cabo **un bloqueo que permita garantizar la consistencia de los datos**.

En la figura 27 se muestra un ejemplo de declaración de un cursor para la actualización del salario.

```
DECLARE
  CURSOR cursor_empleado IS
    SELECT id_empleado, nombre, apellidos, salario
    FROM EMPLEADOS
    WHERE departamentos.id_empleado = empleados.id_empleado
    FOR UPDATE OF salario NOWAIT;
```

Fig.27. Ejemplo de definición de cursor para la actualización de registros.

Como se puede observar, al final de la consulta se utiliza la sentencia '**FOR UPDATE OF** salario**NOWAIT**' que permite **bloquear los registros para modificar el campo salario** sin que otros usuarios puedan cambiarlo mientras tanto.

Una vez definido el cursor como se ha visto en el ejemplo, la apertura se realizaría como ya hemos explicado en el apartado anterior, y se procederá a recorrerlo y actualizar en cada iteración la fila actual.

En la figura 28 se muestra un ejemplo de un bucle que recorrería los resultados obtenidos al ejecutar la consulta que define el cursor y actualizaría el saldo en cada una de ellas.

```
LOOP
  FETCH cursor_empleado INTO v_id_empleado, v_salario;
  EXIT WHEN cursor_empleado%NOTFOUND;

  UPDATE EMPLEADOS
  SET salario = v_salario * 1.1
  WHERE CURRENT OF cursor_empleado;
END LOOP;

CLOSE cursor_empleado;
COMMIT;
```

Fig.28. Ejemplo de bucle recorriendo un cursor para actualizar el salario.

Enlaces de interés...

En este enlace encontrarás la documentación oficial de Oracle relacionada con cursores explícitos. Consultar la documentación oficial te permitirá ampliar tu conocimiento y ganar experiencia en la búsqueda de información facilitada por el fabricante.

/ 12. Caso práctico 1: "Colección de nombres"

Planteamiento: Uno de los clientes más importantes de nuestra empresa es una organización que necesita mejorar la presentación de sus datos. Esta organización ha solicitado una aplicación en PL/SQL que gestione un conjunto de nombres ordenados alfabéticamente y los muestre en orden inverso. Esta funcionalidad es esencial para un informe específico que el cliente necesita generar, y confía en que esto mejorará su eficiencia en la gestión de datos.

Nudo: Para cumplir con esta solicitud, primero debemos definir un tipo de array en PL/SQL que pueda almacenar hasta seis nombres. Luego, declararemos e inicializaremos el array con seis nombres de ciudades españolas ordenados alfabéticamente. Una vez hecho esto, implementaremos un bucle FOR que recorrerá el array en orden inverso, mostrando cada uno de los nombres.

Basándonos en lo anterior, ¿cuáles son las sentencias requeridas para definir e inicializar el array?

Adicionalmente, ¿cuál sería la sentencia requerida para recorrerlo en orden inverso y mostrar su contenido?

Desenlace: Según lo mencionado y lo aprendido en los puntos anteriores, la declaración, la inicialización y el recorrido del array en orden inverso se pueden implementar de la siguiente manera (figura 29):

```
DECLARE
  TYPE t_nombres IS VARRAY(6) OF VARCHAR2(100);
  v_nombres t_nombres := t_nombres('Ana', 'Carlos',
                'Fernando', 'Luis', 'María', 'Pedro');
BEGIN
  -- Recorrer el array en orden inverso
  FOR i IN REVERSE 1..v_nombres.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(v_nombres(i));
  END LOOP;
END;
```

Fig.29. Script que recorre un array en orden inverso y muestra sus datos.

/ 13. Caso práctico 2: "Cursor sobre las noticias"

Planteamiento: Uno de los clientes más importantes de tu empresa es una agencia de noticias, para la cual la información es primordial. A día de hoy, esta agencia está actualizando su base de datos y explorando nuevas formas de gestionar la información, como el uso de *scripts* en base de datos y cursores. Confía en que esto mejorará su eficiencia diaria y aumentará la productividad.

El responsable de IT ha solicitado tu colaboración para crear un cursor que facilite la explotación de la base de datos que vuestra empresa ha implementado y mantiene.

Nudo: El cursor que el cliente desea implementar debe incluir los campos 'id_noticia' y 'titulo' de la tabla 'noticias'.

Basándonos en lo anterior, ¿cuáles son las sentencias requeridas para declarar el cursor?

Adicionalmente, ¿cuál sería la sentencia requerida para abrirlo?

Desenlace: Según lo mencionado y lo aprendido en los puntos anteriores, la declaración de este cursor se corresponde con la mostrada en las figuras 30 y 31.

```
DECLARE cursor_noticias CURSOR FOR
SELECT id_noticia, titulo
FROM noticias;
```

Fig.30. Definición del cursor.

```
OPEN cursor_noticias;
```

Fig.31. Apertura del cursor.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema, hemos examinado las estructuras de datos compuestos esenciales para crear programas robustos en PL/SQL. Primero, hemos analizado el concepto de registro, su declaración y uso, utilizando varios ejemplos para una comprensión más profunda. Luego, hemos estudiado los arrays, incluyendo los asociativos, y las tablas anidadas, revisando ejemplos prácticos de su aplicación.

Finalmente, hemos abordado el concepto de cursor, fundamental para aprovechar la potencia de PL/SQL. Hemos estudiado los tipos de cursores (implícitos y explícitos) y sus diferencias, empleando ejemplos prácticos para ilustrar su uso. Además, hemos explicado el ciclo de vida de los cursores explícitos y cómo se actualizan los registros mediante cursores.

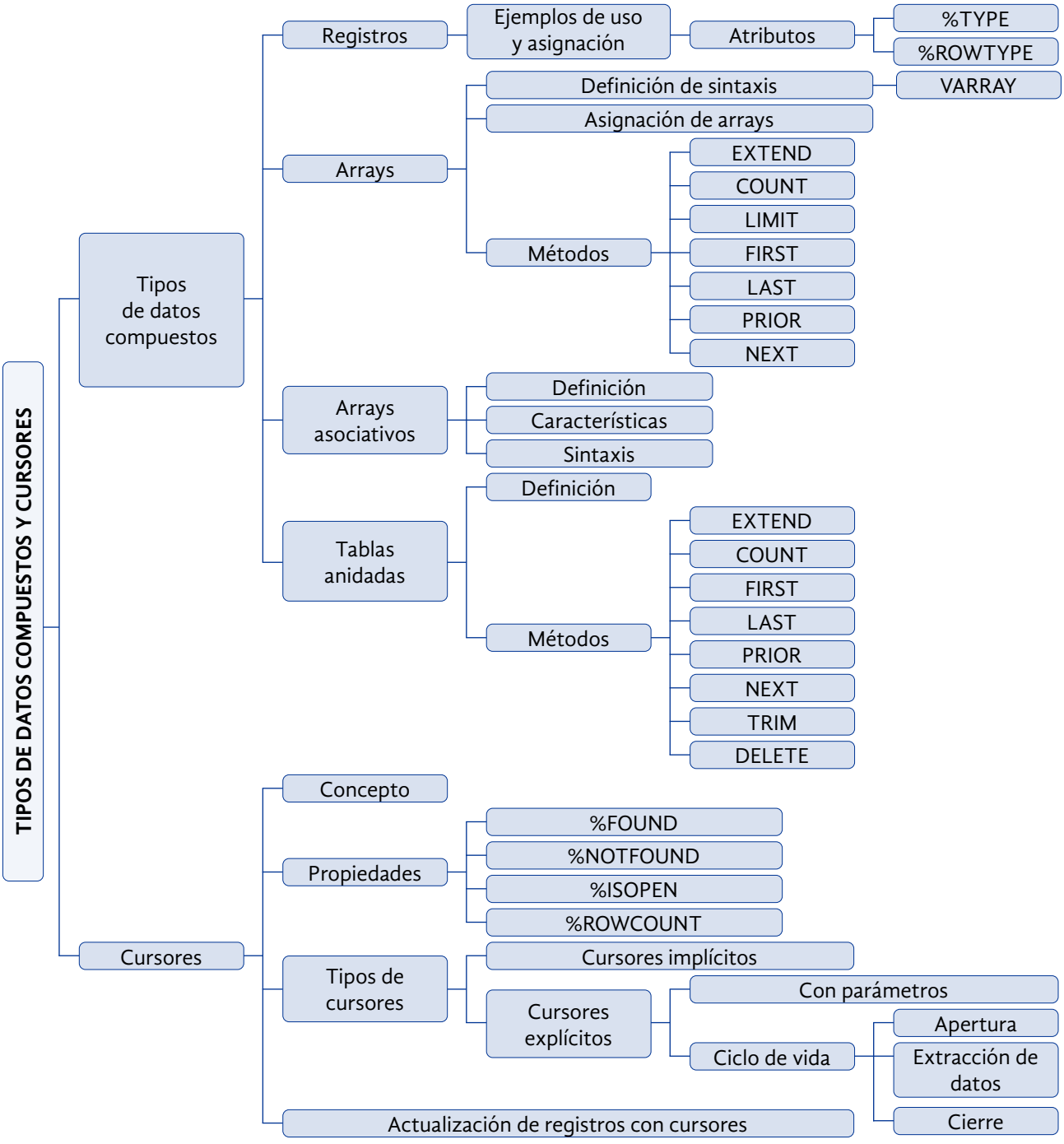


Fig.32. Esquema resumen del tema.

Resolución del caso práctico de la unidad

En el audio introductorio del tema planteamos una serie de preguntas acerca de las estructuras de datos en la programación y su importancia en la gestión eficiente de información.

A continuación, responderemos estas preguntas y exploraremos cómo estas estructuras son fundamentales en el contexto de las bases de datos.

En la primera pregunta, nos referimos a estructuras como registros, tablas anidadas o arrays, que son comunes en la mayoría de los lenguajes de programación. Estas estructuras se utilizan frecuentemente para manejar conjuntos de elementos del mismo tipo, como números, caracteres u objetos, y son esenciales para realizar diversas operaciones.

Para responder a las siguientes dos preguntas, es necesario recordar que, como hemos estudiado, los cursores son estructuras específicas de la programación en bases de datos. Se emplean para almacenar los registros (filas) resultantes de una consulta a la base de datos, permitiendo trabajar y procesar esta información para cumplir con los objetivos deseados. Los cursores son fundamentales en este entorno y pueden compararse a un array de filas de una tabla que cumplen las condiciones especificadas en la consulta.

/ 15. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, Iván y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.